

bok25

基
礎
班

、考研数据結構



二叉树的性质

1. 在二叉树第 i 层上至多有 2^{i-1} ($i \geq 1$) 个节点
2. 深度为 k 的二叉树至多有 $2^k - 1$ ($k \geq 1$) 个节点
3. 对任何一棵二叉树 T ，如果其终端节点数为 n_0 ，度为2的节点数为 n_2 ，则 $n_0 = n_2 + 1$

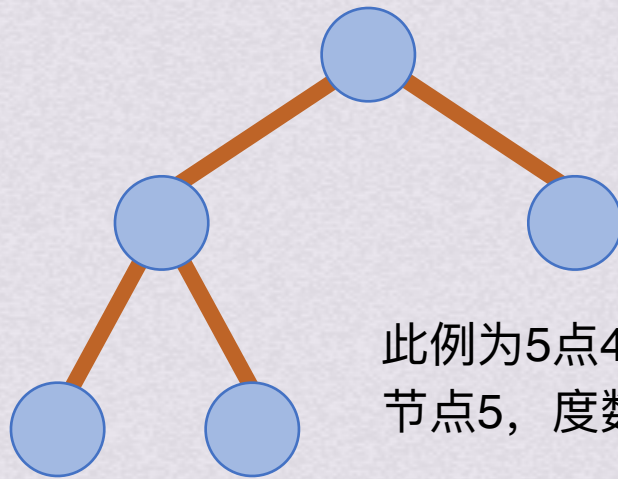
证明过程： 已知：节点数量=度数和+1（点比边多1）

节点数量=度为0节点数+度为1节点数+度为2节点数

度数和= $0 \times$ 度为0节点数+ $1 \times$ 度为1节点数+ $2 \times$ 度为2节点数

即 $n_0 + n_1 + n_2 = 0 \times n_0 + 1 \times n_1 + 2 \times n_2$

化简得 $n_0 = n_2 + 1$

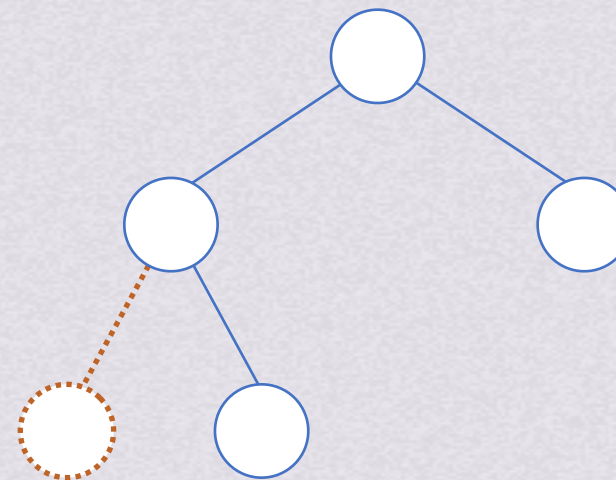


此例为5点4边
节点5，度数和4



特殊形态的二叉树

1. 满二叉树：深度为 k 且含有 2^k-1 个节点的二叉树
(即在深度 k 塞下最多节点)
2. 完全二叉树：深度为 k 的、有 n 个节点的二叉树，当且仅当其每一个节点都与深度为 k 的满二叉树中编号从1至 n 的节点一一对应时，称之为完全二叉树。
(即同深度满二叉树，或者最下层从右往左删除节点以后的满二叉树)



不是完全二叉树!

如果把虚线部分补齐，那就是完全二叉树

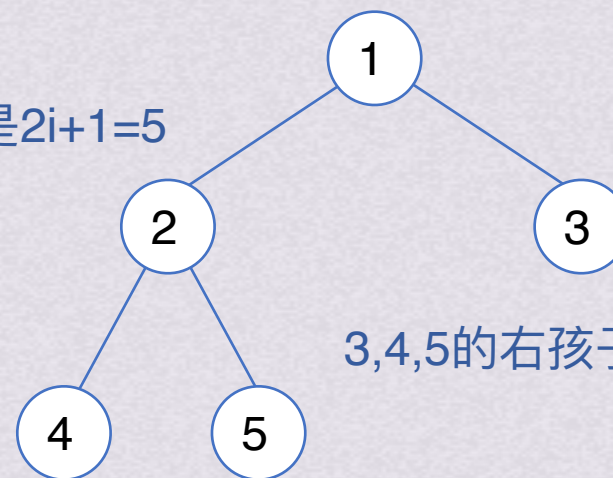


完全二叉树的性质

1. 叶子结点只可能在层次最大的两层出现
2. 对任一节点，若其右分支下的子孙的最大层次为 l ，则其左分支下的子孙的最大层次必为 l 或 $l+1$
3. 具有 n 个节点的完全二叉树的深度为 $\lfloor \log_2 n \rfloor + 1$
4. 如果对一棵有 n 个节点的完全二叉树(其深度为 $\lfloor \log_2 n \rfloor + 1$)的节点按层序编号(从第1层到第 $\lfloor \log_2 n \rfloor + 1$ 层，每层从左到右)，则对任一节点 i ($1 \leq i \leq n$)，以下结论成立。
 - (1) 如果 $i = 1$ ，则节点 i 是二叉树的根，无双亲;如果 $i > 1$ ，则其双亲 $PARENT(i)$ 是节点 $\lfloor i/2 \rfloor$ 。
 - (2) 如果 $2i > n$ ，则节点 i 无左孩子(节点 i 为叶子节点);否则其左孩子 $LCHILD(i)$ 是节点 $2i$ 。
 - (3) 如果 $2i + 1 > n$ ，则节点 i 无右孩子;否则其右孩子 $RCHILD(i)$ 是节点 $2i + 1$ 。

1的右孩子是 $2i+1=3$

2的右孩子是 $2i+1=5$



3,4,5的右孩子是 $2i+1 > 5$ ，不存在



二叉树的存储结构

顺序存储

```
//----- 二叉树的顺序存储表示 -----  
#define MAXTSIZE 100           //二叉树的最大节点数  
typedef TElemType SqBiTree[MAXTSIZE]; //0号单元存储根节点  
SqBiTree bt ;
```

顺序存储只适用于完全二叉树

在最坏的情况下，一个深度为 k 且只有 k 个节点的单支树(树中不存在度为2的节点)却需要长度为 2^k-1 的一维数组。这造成了存储空间的极大浪费，所以**对于一般二叉树，更适合采取链式存储结构**



二叉树的存储结构

链式存储

最少需要包含三个域：数据域、左指针域、右指针域——二叉链表

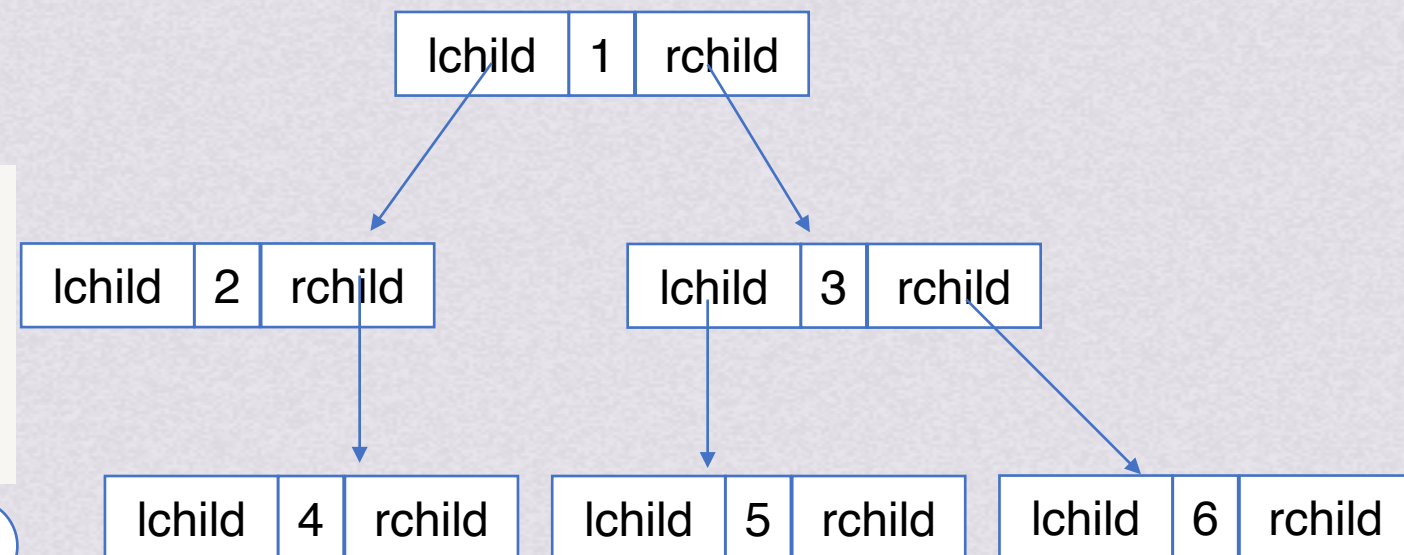
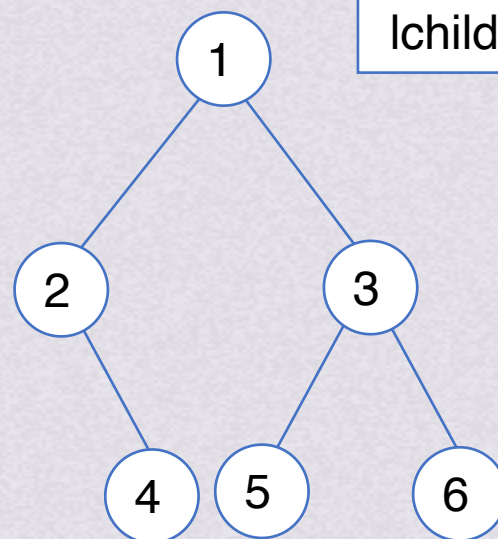
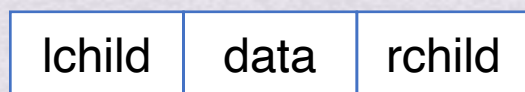
```
//- - - - -二叉树的二叉链表存储表示- - - - -  
typedef struct BiTNode{  
    TElemType data;           //节点数据域  
    struct BiTNode *lchild, *rchild; //左右孩子指针  
}BiTNode, *BiTree;
```

二叉树的存储结构

链式存储

最少需要包含三个域：数据域、左指针域、右指针域——二叉链表

```
//- - - - -二叉树的二叉链表存储表示- - - - -
typedef struct BiTNode{
    TElemType data;           //节点数据域
    struct BiTNode *lchild,*rchild; //左右孩子指针
}BiTNode,*BiTree;
```



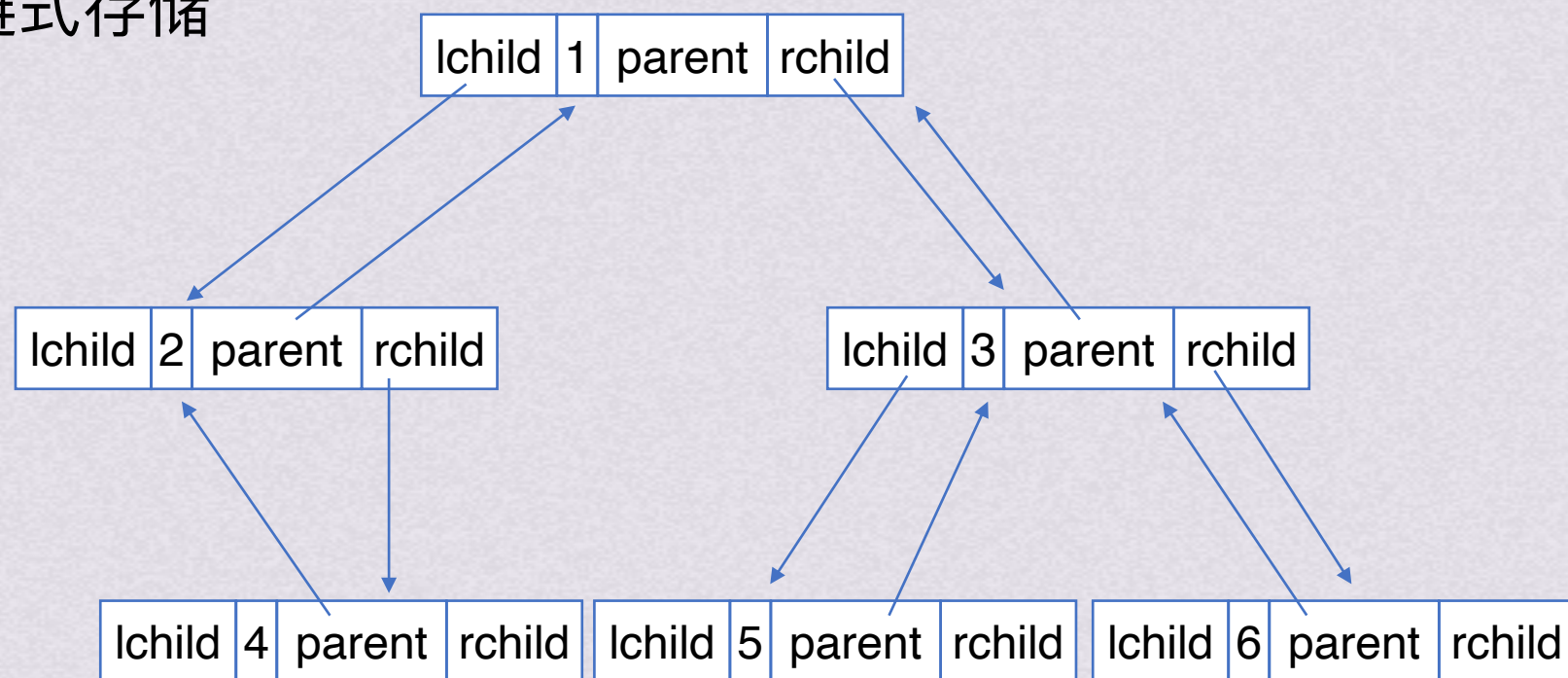
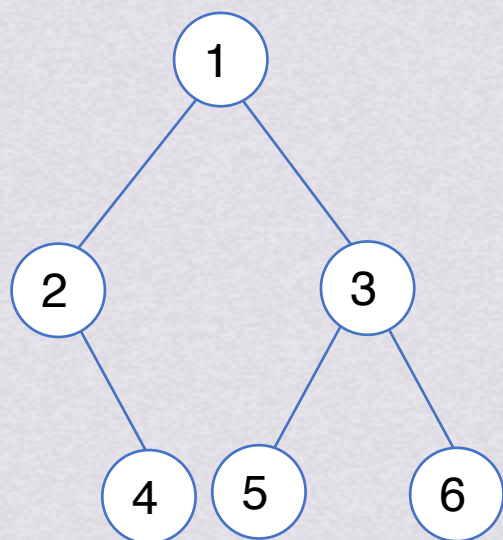
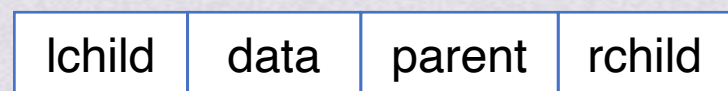


二叉树的存储结构

链式存储

最少需要包含三个域：数据域、左指针域、右指针域——二叉链表

在二叉链表的基础上还可以再添加一个指向双亲节点的指针域，成为三叉链表





遍历二叉树

按某条搜索路径巡访树中每个节点，使得每个节点均被访问一次，而且仅被访问一次



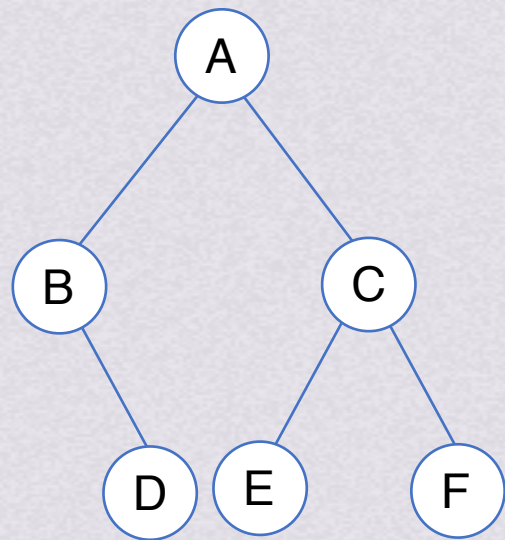
遍历二叉树

遍历二叉树是指按某条搜索路径巡访树中每个节点，使得每个节点均被访问一次，而且仅被访问一次

先序遍历部分 根左右

NLR

```
void PreOrderTraverse(BiTree T)
{
    // 先序遍历二叉树 T 的递归算法
    if(T) {
        // 若二叉树非空
        cout<<T->data; // 访问根节点
        PreOrderTraverse(T->lchild); // 先序遍历左子树
        PreOrderTraverse(T->rchild); // 先序遍历右子树
    }
}
```





遍历二叉树

遍历二叉树是指按某条搜索路径巡访树中每个节点，使得每个节点均被访问一次，而且仅被访问一次

总结

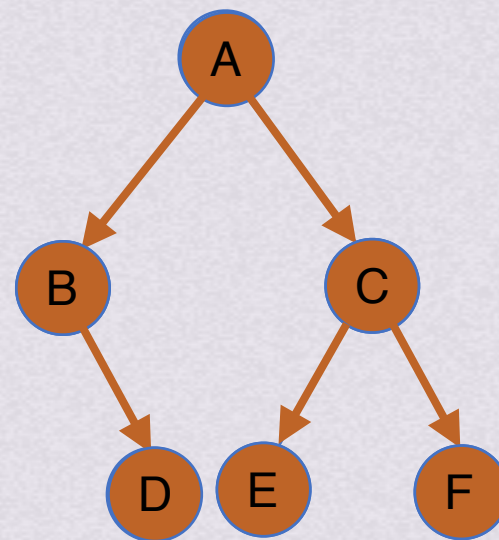
```
void PreOrderTraverse(BiTree T)
{
    // 先序遍历二叉树 T 的递归算法
    if(T) {
        // 若二叉树非空
        cout<<T->data; // 访问根节点
        PreOrderTraverse(T->lchild); // 先序遍历左子树
        PreOrderTraverse(T->rchild); // 先序遍历右子树
    }
}
```

规律1.第一次经过某个节点时就输出它。输出后，就去访问它的左孩子。

规律2.访问左孩子的过程中如果访问到空，则按照访问路径退回，每退回一个输出过的节点，如果该节点的右孩子还没有被访问，就去访问它的右孩子。

规律3.访问右孩子的过程中如果访问到空，则按照访问路径退回到上一个节点。

规律4.第三次经过某个节点时（即该节点的左右孩子都已输出），代表该节点已处理完毕，按照访问路径退回到上一个节点。



输出序列: A B D C E F



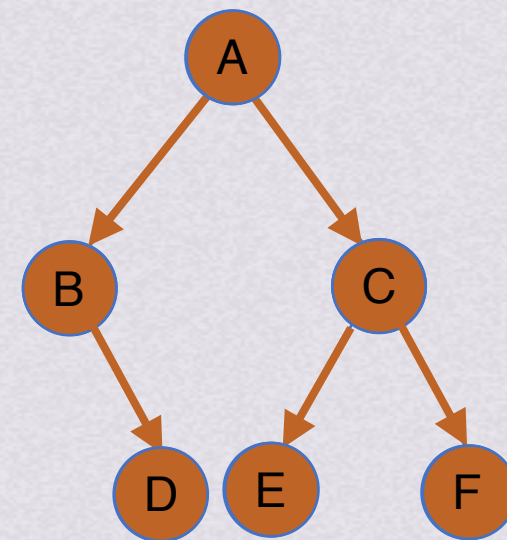
遍历二叉树 遍历二叉树是指按某条搜索路径巡访树中每个节点，使得每个节点均被访问一次，而且仅被访问一次

先序遍历序列 根左右

中序遍历部分

A B D C E F NLR

```
void InOrderTraverse(BiTree T)
{
    // 中序遍历二叉树 T 的递归算法
    if(T) {
        // 若二叉树非空
        InOrderTraverse(T->lchild); // 中序遍历左子树
        cout<<T->data; // 访问根节点
        InOrderTraverse(T->rchild); // 中序遍历右子树
    }
}
```



中序遍历输出序列: B D A E C F

递归栈



遍历二叉树 遍历二叉树是指按某条搜索路径巡访树中每个节点，使得每个节点均被访问一次，而且仅被访问一次

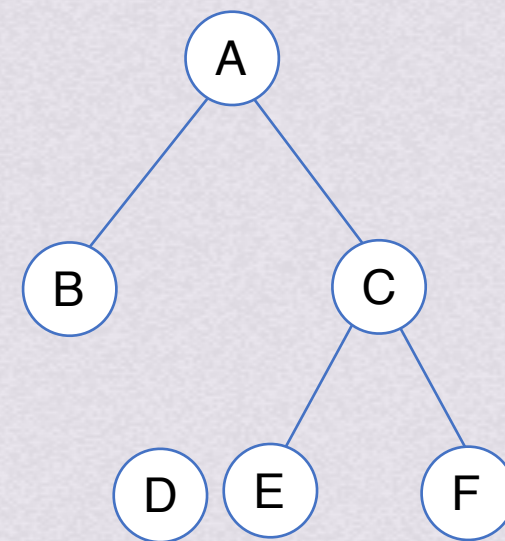
先序遍历序列 根左右

A B D C E F NLR

中序遍历序列 左根右

B D A E C F LNR

以NLR的顺序依次遍历二叉树，得到了先序遍历序列

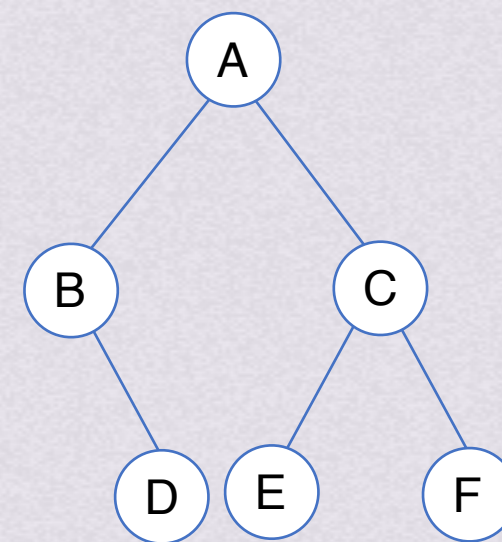




遍历二叉树 遍历二叉树是指按某条搜索路径巡访树中每个节点，使得每个节点均被访问一次，而且仅被访问一次

先序遍历序列	根左右	中序遍历序列	左根右	后序遍历部分	左右根
A B D C E F	NLR	B D A E C F	LNR	LRN	

```
void PostOrderTraverse(BiTree T)
{
    // 后序遍历二叉树 T 的递归算法
    if(T) {
        // 若二叉树非空
        PostOrderTraverse(T->lchild); // 后序遍历左子树
        PostOrderTraverse(T->rchild); // 后序遍历右子树
        cout<<T->data; // 访问根节点
    }
}
```



递归栈



遍历二叉树 遍历二叉树是指按某条搜索路径巡访树中每个节点，使得每个节点均被访问一次，而且仅被访问一次

先序遍历序列 根左右

中序遍历序列 左根右

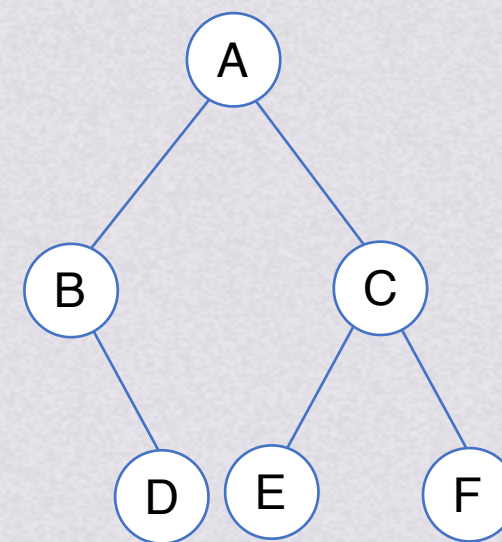
A B D C E F NLR

B D A E C F LNR

后序遍历序列 左右根

D B E F C A LRN

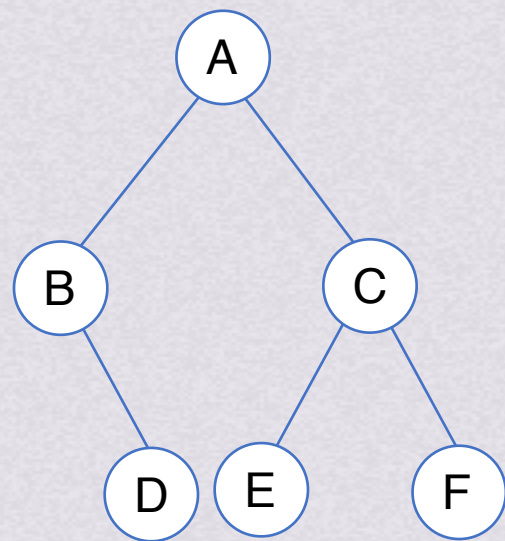
以LRN的顺序依次遍历二叉树，得到了先序遍历序列





遍历二叉树

遍历二叉树是指按某条搜索路径巡访树中每个节点，使得每个节点均被访问一次，而且仅被访问一次



先序遍历序列 根左右

A B D C E F NLR

中序遍历序列 左根右

B D A E C F LNR

后序遍历序列 左右根

D B E F C A LRN



遍历二叉树代码再回顾

遍历二叉树是指按某条搜索路径巡访树中每个节点，使得每个节点均被访问一次，而且仅被访问一次

先序遍历
根左右
NLR

```
void PreOrderTraverse(BiTree T) {  
    // 先序遍历二叉树 T 的递归算法  
    if(T)//若二叉树非空  
    {  
        cout<<T->data;    // 访问根节点  
        PreOrderTraverse(T->lchild); // 先序遍历左子树  
        PreOrderTraverse(T->rchild); // 先序遍历右子树  
    }  
}
```

中序遍历
左根右
LNR

```
void InOrderTraverse(BiTree T) {  
    // 中序遍历二叉树 T 的递归算法  
    if(T)//若二叉树非空  
    {  
        InOrderTraverse(T->lchild); // 中序遍历左子树  
        cout<<T->data;    // 访问根节点  
        InOrderTraverse(T->rchild); // 中序遍历右子树  
    }  
}
```

后序遍历
左右根
LRN

```
void PostOrderTraverse(BiTree T) {  
    // 后序遍历二叉树 T 的递归算法  
    if(T)//若二叉树非空  
    {  
        PostOrderTraverse(T->lchild); // 后序遍历左子树  
        PostOrderTraverse(T->rchild); // 后序遍历右子树  
        cout<<T->data;    // 访问根节点  
    }  
}
```



遍历二叉树

遍历二叉树是指按某条搜索路径巡访树中每个节点，使得每个节点均被访问一次，而且仅被访问一次

层次遍历：按照第一层，第二层...这样的顺序遍历所有节点

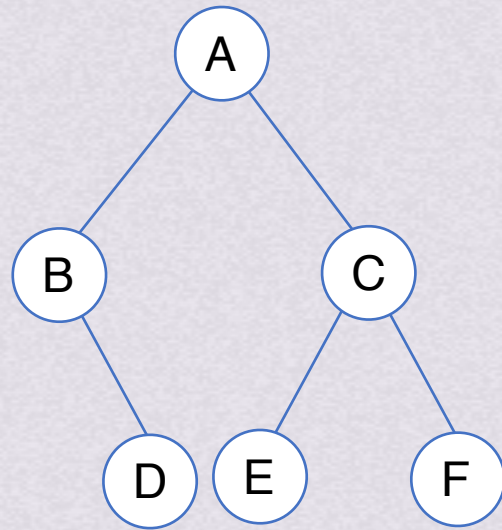


遍历二叉树

代码运行模拟

层次遍历：按照第一层，第二层...这样的顺序遍历所有节点

```
void LevelOrder(BiTree T){  
    //以层为顺序遍历树  
    InitQueue(Q); //初始化辅助队列  
    BiTree p;  
    EnQueue(Q,T); //将根节点入队  
    while(!IsEmpty(Q)){ //队列不空则循环  
        DeQueue(Q,p); //队头结点出队  
        visit(p); //访问出队节点  
        if(p->lchild!=NULL)  
            EnQueue(Q,p->lchild); //左孩子不空，则左孩子入队  
        if(p->rchild!=NULL)  
            EnQueue(Q,p->rchild); //右孩子不空，则右孩子入队  
    }  
}
```



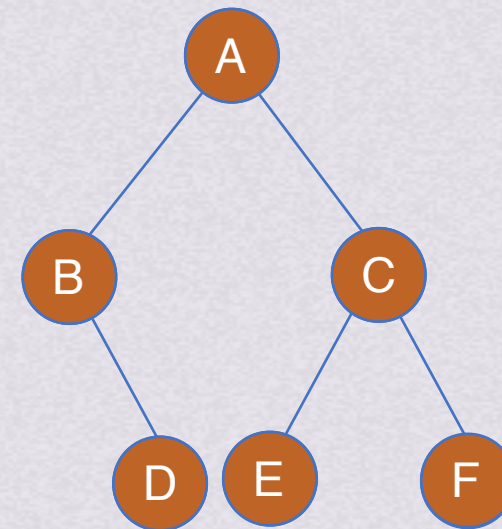


遍历二叉树

代码运行模拟

层次遍历：按照第一层，第二层...这样的顺序遍历所有节点

```
void LevelOrder(BiTree T){ 传入树根节点A
//以层为顺序遍历树
    InitQueue(Q); //初始化辅助队列
    BiTree p;
    EnQueue(Q,T); //将根节点入队 A节点入队
    while(!IsEmpty(Q)){ //队列不空则循环 循环进入条件不满足,
        DeQueue(Q,p); //队头结点出队 不再进入循环
        visit(p); //访问出队节点
        if(p->lchild!=NULL)
            EnQueue(Q,p->lchild); //左孩子不空, 则左孩子入队
        if(p->rchild!=NULL)
            EnQueue(Q,p->rchild); //右孩子不空, 则右孩子入队
    }
}
```



层序遍历序列: A B C D E F

节点A

节点B

节点C

节点D

节点E

节点F

p↑

← 队头

辅助队列

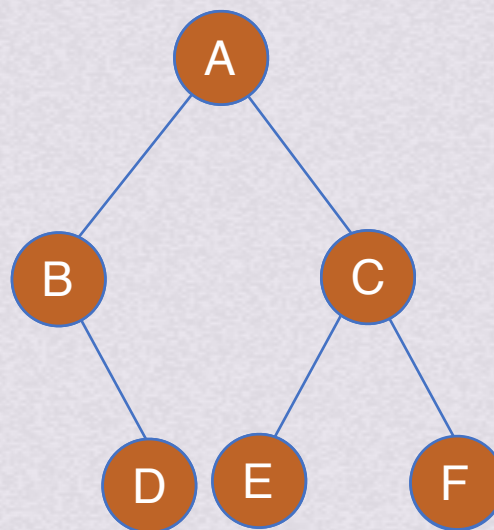
队尾 →



遍历二叉树

代码运行模拟

层次遍历：按照第一层，第二层...这样的顺序遍历所有节点



层序遍历序列：A B C D E F

节点A

节点B

节点C

节点D

节点E

节点F



由遍历序列确认二叉树

已知前序遍历序列，中序遍历序列，后序遍历序列中的任意一个的时候是无法确定一棵二叉树的

但已知中序遍历序列+其他三种遍历序列的任意一种，即可确定唯一的一棵二叉树

那么怎样才能求出来呢？

先序遍历输出序列:1 2 4 3 5 6 中序遍历输出序列:2 4 1 5 3 6

先序遍历：根左右

中序遍历：左根右

由遍历序列确认二叉树

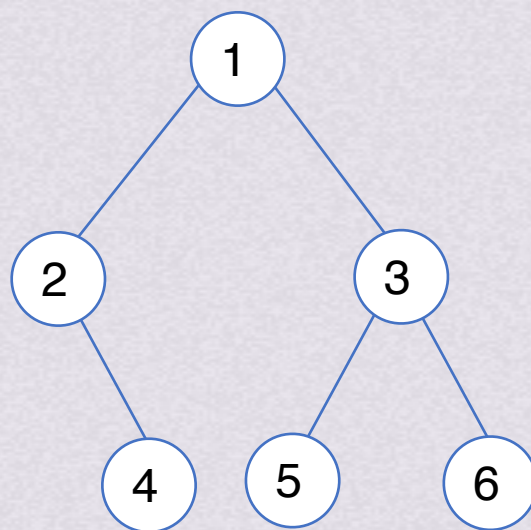
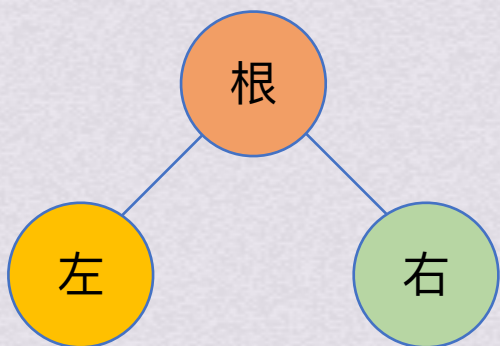
先序遍历输出序列: 1 2 4 3 5 6

先序遍历: 根左右

中序遍历输出序列: 2 4 1 5 3 6

中序遍历: 左根右

最后再来看536, 从先序仍能看出3为根, 56为孩子



536对应的左根右有三种可能

53为左子树, 6为根

5 3 6
左 根 右

5为根, 36为右子树

5 3 6
左 根 右

5为左子树, 3为根, 6为右子树

5 3 6
左 根 右

其中只有5,3,6分别为左、根、右满足3为根



小试牛刀

(2011) 一棵二叉树的前序遍历序列和后序遍历序列分别为1,2,3,4和4,3,2,1, 该二叉树的中序遍历序列不会是 ()

A.1,2,3,4

B.2,3,4,1

C.3,2,4,1

D.4,3,2,1

小试牛刀

(2011) 一棵二叉树的前序遍历序列和后序遍历序列分别为1,2,3,4和4,3,2,1, 该二叉树的中序遍历序列不会是 ()

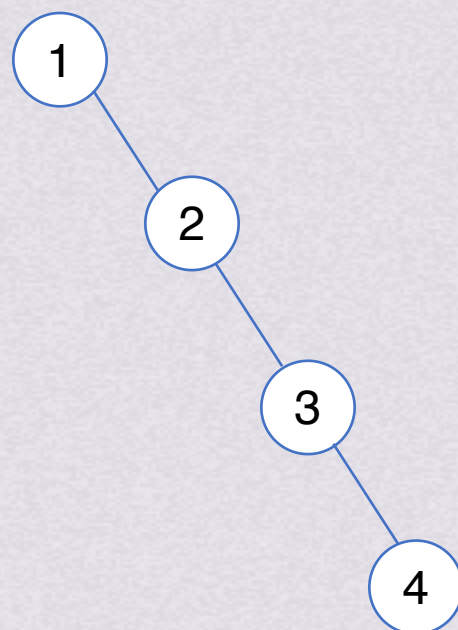
A.1,2,3,4

B.2,3,4,1

C.3,2,4,1

D.4,3,2,1

对于A, 如果前序遍历序列和中序遍历序列都是1,2,3,4



前序: 根左右
1 2 3 4

中序: 左根右
1 2 3 4

该树的前序遍历序列1,2,3,4

中序遍历序列1,2,3,4

后序遍历序列是4,3,2,1, A不符合题意, 排除